Data Glacier

Data Science Intern at Data Glacier

Week 4: Deployment on Flask

Name: Mehedi Hasan Shakil

Batch Code: LISUM44

Date: 26 April 2025

Submitted to: Data Glacier

Table of Contents:

1. Introduction

In this project, we are going to deploy a machine learning model (Random Forest) using the Flask Framework. As a demonstration, our model helps to predict fake and genuine accounts on Instagram.

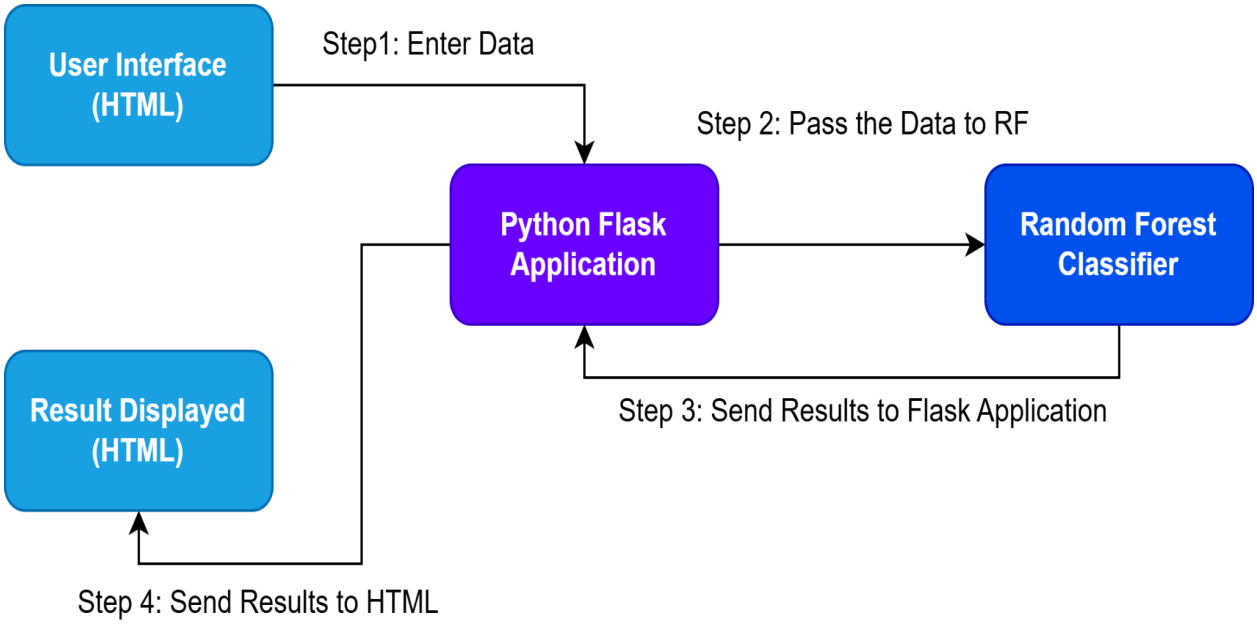


Figure 1.1: Application Workflow

We will focus on both: building a machine learning model for a fake instagram account detector, then creating an API for the model, using Flask, the Python micro-framework for building web applications. This API allows us to utilize predictive capabilities through HTTP requests.

2. Data Information

The dataset used in this project is a curated collection of Instagram account data labeled as either Fake (1) or Genuine (0). It consists of 696 records, each representing an Instagram profile with 11 numerical features extracted from account metadata. This dataset is commonly used for binary classification tasks to train models capable of detecting potentially fake accounts based on their profile characteristics. The table below describes the dataset.

Table 2.1: Dataset Information

Description	Value
Total Records	696
Total Fake Account Records	383
Total Genuine Account Records	313
Total features	11
Target Variable	fake (1 = Fake, 0 = Genuine)

2.1 Attribute Information

The collection is composed of two CSV file, where each line has the following attributes:

Table 2.2: Attribute Information

Feature Name	Description
profile pic	Binary: 1 if profile picture exists, 0 otherwise
nums/length username	Ratio of digits to the total username length
fullname words	Number of words in the fullname
nums/length fullname	Ratio of digits to the total fullname length
name==username	Binary: 1 if name and username are the same, 0 otherwise
description length	Total number of characters in the bio
external url	Binary: 1 if an external link is present, 0 otherwise
private	Binary: 1 if the account is private, 0 if public
#posts	Number of posts shared on the account
#followers	Number of followers
#follows	Number of accounts the user follows

3. Build Machine Learning Model

3.1.1 Import the Dataset

In this part, we import the datasets and merge them which contains the information of 693 instagram account features.

```
# Mount Google Drive
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
1. Load the Dataset
```

```
[ ] # Load train and test datasets
    import pandas as pd

    train = pd.read_csv('/content/drive/MyDrive/Flask deployment/train.csv')
    test = pd.read_csv('/content/drive/MyDrive/Flask deployment/test.csv')
```

```
[ ] # Merge datasets
    df = pd.concat([train, test], ignore_index=True)

    #Preview
    print("Dataset shape:", df.shape)
    df.head()
```

3.1.1 Data Preprocessing

The dataset used here is split into 80% for the training set and the remaining 20% for the test set.

```
2. Preprocess the Dataset
```

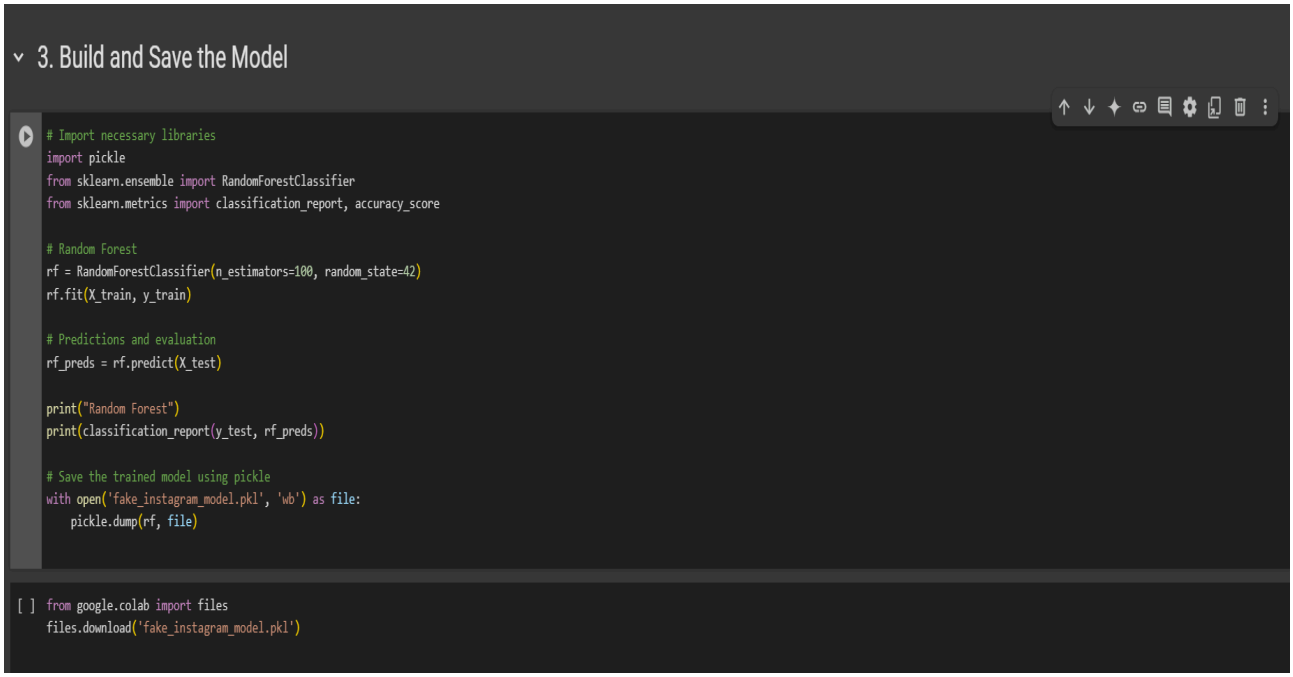
```
[ ] # Check missing values and data types
    df.info()
    df.isnull().sum()
```

```
# Distribution of target
import seaborn as sns
import matplotlib.pyplot as plt

sns.countplot(x='fake', data=df)
plt.title("Fake vs Real Account Distribution")
plt.show()
```

```
[ ] # Define features and target
    X = df.drop('fake', axis=1)
    y = df['fake']

    # Split the data into training and testing
    from sklearn.model_selection import train_test_split

    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
```

3.1.2 Build and Save Model

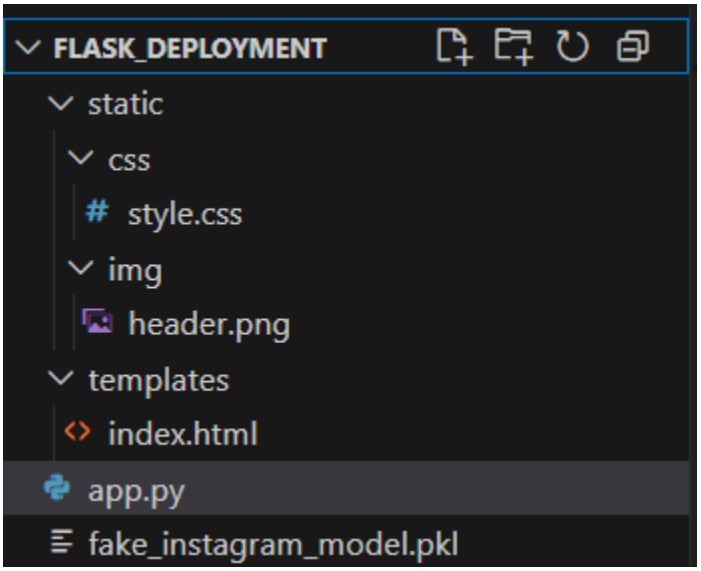
After data preprocessing, we implement a machine learning model to classify the fake instagram accounts. For this purpose, we implement Random Forest (RF) using scikit-learn. After importing and initializing the RF model we fit into the training dataset and then saved our model using pkl.



4. Turning Model into Web Application

We develop a web application that consists of a simple web page with a form field that lets us enter account features. After submitting the features to the web application, it will give us a result of fake or genuine.

First, we create a folder for this project called FLASK_DEPLOYMENT, this is the directory tree inside the folder. We will explain each file.



The sub-directory templates are the directory in which Flask will look for static HTML files for rendering in the web browser, in our case, we have one HTML file: *index.html*.

4.1 App.py

The *app.py* file contains the main code that will be executed by the Python interpreter to run the Flask web application, it includes the ML code for classifying fake instagram accounts.

app.py > ...

You, 54 seconds ago | 1 author (You)

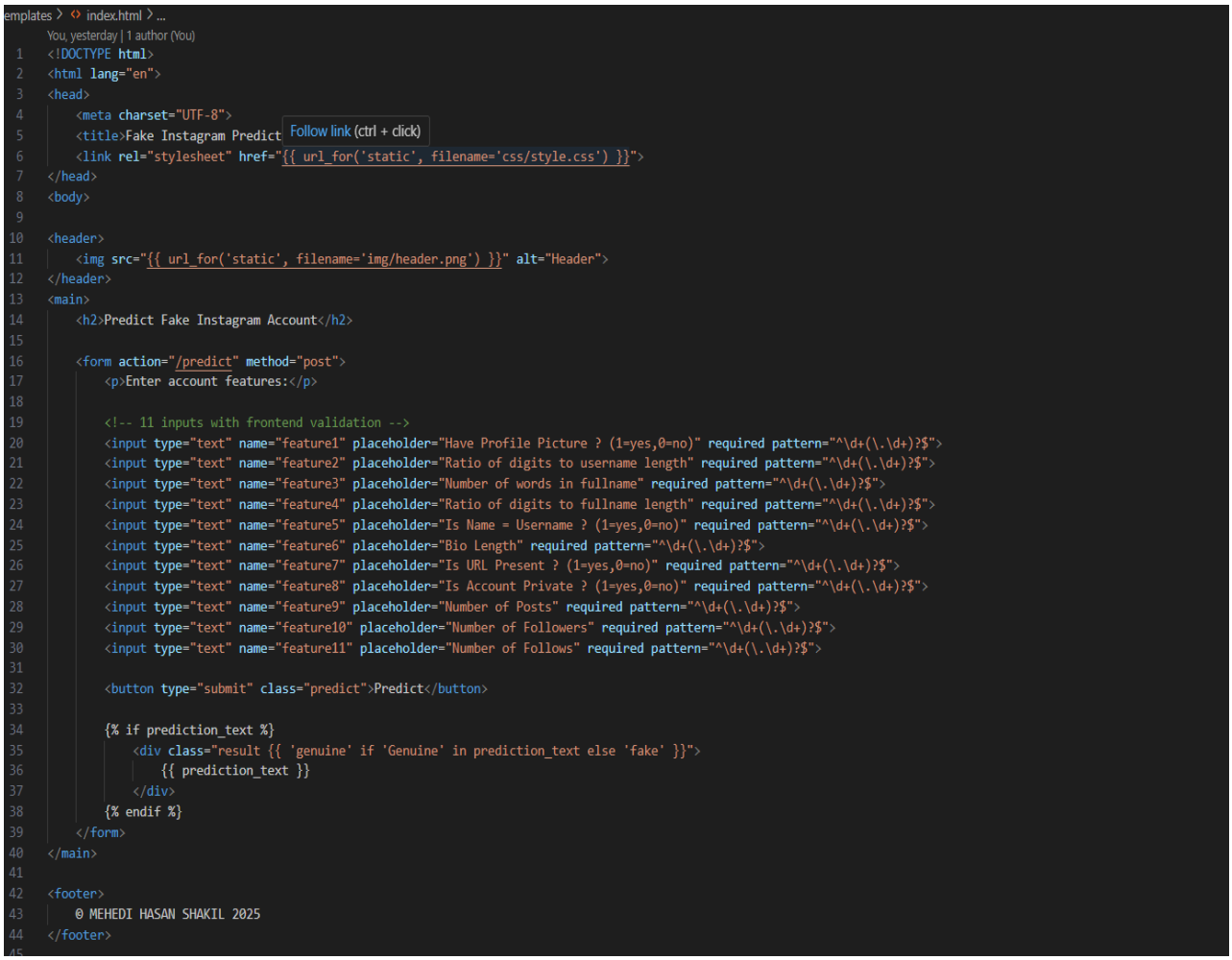
```python
from flask import Flask, render_template, request
import pickle
import numpy as np

# Load the trained model
with open('fake_instagram_model.pkl', 'rb') as f:
    rf = pickle.load(f)

app = Flask(__name__)

@app.route('/')
def home():
    return render_template('index.html')

@app.route('/predict', methods=['POST'])
def predict():
    # Try to grab the input features and convert to float
    try:
        features = [float(x) for x in request.form.values()]
    except ValueError:
        # If there is an error in conversion, return the same template with an error message
        return render_template('index.html', prediction_text="Please enter valid numeric values for all features.")

    # Check if the correct number of features is entered (11 features)
    if len(features) != 11:
        return render_template('index.html', prediction_text="Please make sure to enter all 11 features.")

    # Convert features to numpy array for prediction
    final_features = np.array(features).reshape(1, -1)

    # Model prediction (0 or 1)
    raw_pred = rf.predict(final_features)[0]

    # Map to friendly text
    if raw_pred == 0:
        message = "The account is Genuine ✅"
    else:
        message = "The account is Fake ❌"

    # Send the result back to the page
    return render_template('index.html', prediction_text=message)

if __name__ == '__main__':
    app.run(debug=True)
```

Figure 4.1: App.py

- We ran our application as a single module; thus we initialized a new Flask instance with the argument *_name_* to let Flask know that it can find the HTML template folder (*templates*) in the same directory where it is located.
- Next, we used the route decorator *(@app.route('/'))* to specify the URL that should trigger the execution of the home function.
- Our *home* function simply rendered the *index.html* HTML file, which is located in the *templates* folder.
- Inside the *predict* function, we access the fake instagram data set, pre-process the text, and make predictions, then store the model. We access the features entered by the user and use our model to make a prediction for its label.
- Lastly, we used the *run* function to only run the application on the server when this script is directly executed by the Python interpreter, which we ensured using the *if* statement with *name_== '_main*

4.2 Index.html

The following are the contents of the *index.html* file that will render a form where a user can enter the account features.



```
emplates > index.html > ...
      You, yesterday | 1 author (You)
1     <!DOCTYPE html>
2     <html lang="en">
3     <head>
4         <meta charset="UTF-8">
5         <title>Fake Instagram Predict</title>
6         <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
7     </head>
8     <body>
9
10    <header>
11        <img src="{{ url_for('static', filename='img/header.png') }}" alt="Header">
12    </header>
13    <main>
14        <h2>Predict Fake Instagram Account</h2>
15
16        <form action="/predict" method="post">
17            <p>Enter account features:</p>
18
19            <!-- 11 inputs with frontend validation -->
20            <input type="text" name="feature1" placeholder="Have Profile Picture ? (1=yes,0=no)" required pattern="^\d+(\.\d+)?$">
21            <input type="text" name="feature2" placeholder="Ratio of digits to username length" required pattern="^\d+(\.\d+)?$">
22            <input type="text" name="feature3" placeholder="Number of words in fullname" required pattern="^\d+(\.\d+)?$">
23            <input type="text" name="feature4" placeholder="Ratio of digits to fullname length" required pattern="^\d+(\.\d+)?$">
24            <input type="text" name="feature5" placeholder="Is Name = Username ? (1=yes,0=no)" required pattern="^\d+(\.\d+)?$">
25            <input type="text" name="feature6" placeholder="Bio Length" required pattern="^\d+(\.\d+)?$">
26            <input type="text" name="feature7" placeholder="Is URL Present ? (1=yes,0=no)" required pattern="^\d+(\.\d+)?$">
27            <input type="text" name="feature8" placeholder="Is Account Private ? (1=yes,0=no)" required pattern="^\d+(\.\d+)?$">
28            <input type="text" name="feature9" placeholder="Number of Posts" required pattern="^\d+(\.\d+)?$">
29            <input type="text" name="feature10" placeholder="Number of Followers" required pattern="^\d+(\.\d+)?$">
30            <input type="text" name="feature11" placeholder="Number of Follows" required pattern="^\d+(\.\d+)?$">
31
32            <button type="submit" class="predict">Predict</button>
33
34            {% if prediction_text %}
35                <div class="result {{ 'genuine' if 'Genuine' in prediction_text else 'fake' }}">
36                    {{ prediction_text }}
37                </div>
38            {% endif %}
39        </form>
40    </main>
41
42    <footer>
43        © MEHEDI HASAN SHAKIL 2025
44    </footer>
```

Figure 4.2: index.html

4.3 Style.css

In the header section of *index.html*, we loaded *styles.css* file. CSS is to determine how the look and feel of HTML documents. *styles.css* has to be saved in a subdirectory called *static*, which is the default directory where Flask looks for static files such as CSS.

4.4 Running Procedure

Once we have done all of the above, we can start running the API by either double click *app.py*, or executing the command from the Terminal:

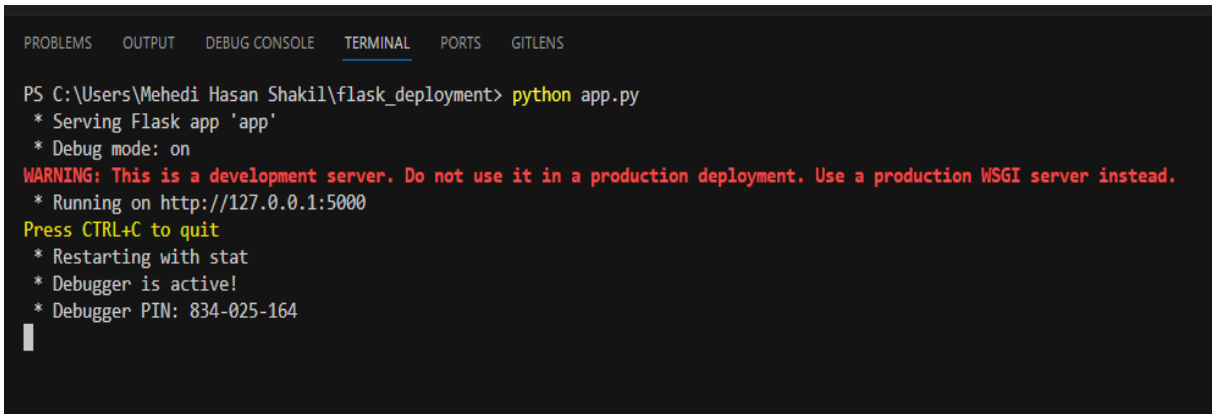


Figure 4.3: Command Execution

Now we could open a web browser and navigate to http://127.0.0.1:5000/, we should see a simple website with the content like so

Figure 4.4: Fake Instagram Account Detection Website Page

Now we enter input in the comments form

Figure 4.5: Input In The Comments Form

After entering the input click the predict button now, we can see the result of our input.

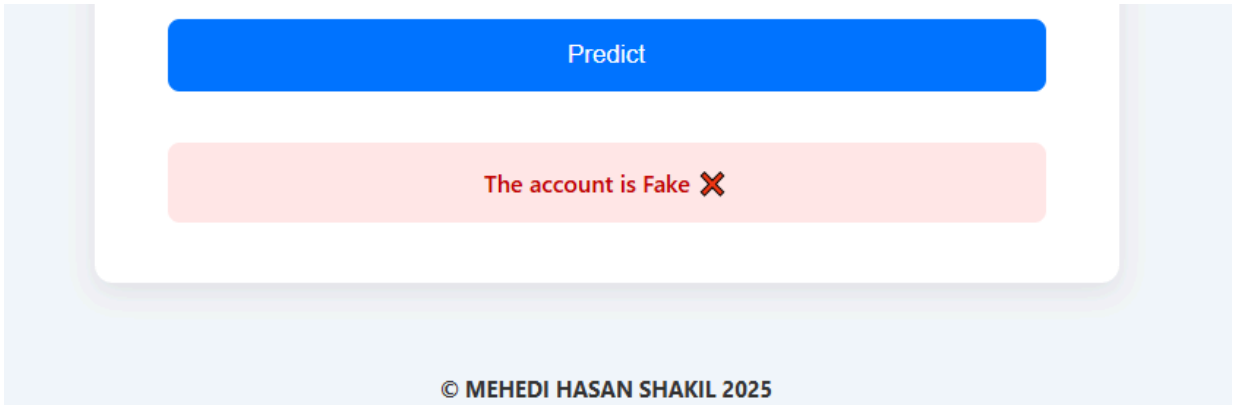


Figure 4.6: Result of Given Input